Machine Learning Tutorial: Demystifying Subword Tokenization – From BPE to Byte-Level Processing

Student ID: 24113860

Topic: Natural Language Processing (NLP) / Advanced Tokenization

Source code: link

1. Introduction: The Gateway to Natural Language Processing

Before a machine learning model can understand human text, the raw text must be converted into numbers. This foundational step is known as tokenization. While modern architectures like Transformers receive the spotlight for their predictive capabilities, the unsung hero of Natural Language Processing (NLP) is the tokenizer. If a model is fed poorly tokenized data, its downstream performance will severely degrade, regardless of the neural network's complexity.

This tutorial explores the algorithmic evolution of tokenization. We will move beyond elementary word and character splitting, diving deep into the mathematics and implementation of Subword Tokenization, specifically focusing on Byte-Pair Encoding (BPE) and Byte-level BPE (BBPE). By the end of this guide, you will understand how modern Large Language Models (LLMs) elegantly handle infinite vocabularies and multi-lingual datasets without ever encountering an "Unknown" word.

2. The Tokenization Dilemma: Words vs Characters

To appreciate the elegance of subword algorithms, we must first understand why the two most intuitive methods—Word-based and Character-based tokenization—fail in complex, real-world machine learning environments.

2.1 The Pitfalls of Word-Based Tokenization

The most straightforward approach is to split sentences by spaces and punctuation. For example, the sentence "The data scientist is coding" becomes `["The", "data", "scientist", "is", "coding"]`.

However, this creates a massive, unwieldy vocabulary. In English alone, adding morphological variations (e.g., code, coding, coded, coder) creates distinct, independent

tokens. The model fails to recognize the semantic relationship between "code" and "coding." More critically, word-based tokenizers suffer from the **Out-Of-Vocabulary (OOV)** problem. If the model encounters a rare or misspelled word during inference that was not in the training data, it maps it to a generic `<UNK>` (unknown) token, instantly losing crucial contextual information.

2.2 The Inefficiency of Character-Based Tokenization

To solve the OOV problem, we could tokenize text into individual characters. The vocabulary size shrinks dramatically (e.g., 26 English letters plus punctuation). Every possible word can be constructed from these base characters, completely eliminating the `<UNK>` token.

Unfortunately, this introduces two fatal flaws for machine learning:

1. **Loss of Semantic Meaning:** A single character like "c" carries no inherent semantic meaning compared to the word "code." The model must work much harder, relying heavily on deep neural layers, to learn that a specific sequence of characters forms a meaningful concept.
2. **Exploding Sequence Lengths:** A 10-word sentence might become a 50-token sequence. Because models like Transformers scale quadratically with sequence length (due to the self-attention mechanism), character-based tokenization demands astronomical computational resources and Memory (FLOPS).

[IMAGE PLACEHOLDER 1: A flowchart or visual table comparing the sequence length and vocabulary size of Word vs. Character tokenization. *-> Ask me how to design this chart to make it visually striking!*]

3. The Sweet Spot: Subword Tokenization

Subword tokenization bridges the gap. It operates on a simple but powerful heuristic: **frequent words should be kept as single, whole tokens, while rare words should be broken down into meaningful subword units.**

For example, the rare word "tokenization" might be split into `["token", "ization"]`.

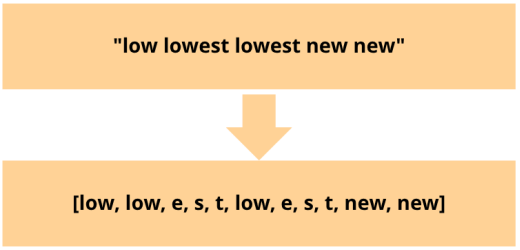
- "token" is a frequent base word.
- "ization" is a frequent suffix.

By combining these known subwords, the model retains semantic meaning, keeps the vocabulary size manageable, and handles completely unseen words without relying on `<UNK>`. The most prominent algorithm for achieving this is Byte-Pair Encoding (BPE) (Sennrich, Haddow and Birch, 2015).

4. Deep Dive: Byte-Pair Encoding (BPE) Algorithm

Byte-Pair Encoding (BPE) was originally introduced in 1994 as a data compression technique (Gage, 1994), but it has since been ingeniously adapted for Natural Language Processing (NLP) (Sennrich, Haddow and Birch, 2015). Unlike traditional tokenizers that blindly split text by spaces, BPE is a bottom-up, iterative algorithm that relies on statistical frequency to mathematically determine the optimal subword vocabulary.

Let us walk through the exact mechanics of training a BPE tokenizer using a sample corpus depend on below example



corpus		vocabulary
5	l o w </w>	'</w>', 'l', 'o', 'w', 'e', 's', 't', 'n', 'r', 'i', 'd', 'er', 'er</w>', 'ne', 'new', 'lo', 'low', 'newer</w>, 'low</w>' len: 20
2	l o w e s t </w>	
6	n e w e r </w>	
3	w i d e r </w>	
2	n e w </w>	

Step 1: Pre-tokenization and Initialization

First, the training corpus is pre-tokenized into standard words (typically using a simple space-based split). We append a special end-of-word symbol—in this case, `</w>`—to every word. This is crucial because it helps the algorithm distinguish between a subword at the end of a word (like "er" in "newer") versus a subword at the beginning of a word (like "er" in "error").

The initial vocabulary is strictly the set of all unique base characters present in the entire corpus.

- **Base Vocabulary:** `</w>`, `l`, `o`, `w`, `e`, `s`, `t`, `n`, `r`, `i`, `d` (11 base tokens).

Step 2: Frequency Counting

Next, we count the exact frequency of every word in our corpus and represent them as sequences of our base character vocabulary.

Step 3: Iterative Merging

The core engine of BPE is the **merge operation**. The algorithm scans the corpus to find the most frequently adjacent pair of tokens and merges them into a single, new token. This repeats for a pre-defined number of iterations.

Let's trace the first few merges that build our vocabulary:

- **Iteration 1 & 2 (Forming the suffix 'er'):**
  Scanning the sequences, the algorithm notices that the tokens e and r appear adjacent to each other 9 times (6 times in "newer" + 3 times in "wider"). This is the highest frequency pair in the corpus.
  - **Action:** Merge e + r -> er
  - **Action:** Immediately after, er and </w> appear together 9 times. Merge er + </w> -> er</w>.
  - *The tokenizer has now mathematically learned that "er" is a highly common word-ending suffix!*
- **Iteration 3 & 4 (Forming 'new'):**
  Next, it looks at the remaining letters. The tokens n and e appear together 8 times (6 in "newer" + 2 in "new").
  - **Action:** Merge n + e -> ne.
  - Next, ne and w appear together 8 times.
  - **Action:** Merge ne + w -> new.
- **Iteration 5 & 6 (Forming 'low'):**
  Following the same logic, the tokens l and o appear 7 times (5 in "low" + 2 in "lowest").
  - **Action:** Merge l + o -> lo.
  - Next, lo and w appear 7 times.
  - **Action:** Merge lo + w -> low.

By continually applying this greedy merging strategy, the algorithm expands its vocabulary beyond single characters to include frequent subwords and entire words. As seen in our final vocabulary list, we have learned distinct tokens like new, low, and newer</w>.

Step 4: Tokenizing Unseen Text (The Application)

Now that our tokenizer is trained, how does it process a sequence of text like `"low lowest lowest new new"`?

The tokenizer breaks down the string using the longest matching tokens it learned during training:

1. **"low"**: Matches our learned token `low` perfectly.
2. **"lowest"**: The algorithm recognizes the subword `low`. However, because the letters `e`, `s`, and `t` only appeared together 2 times in our training corpus, the algorithm never reached a high enough iteration to merge them into an `est` token. Therefore, it falls back to the base characters: `e`, `s`, `t`.
3. **"new"**: Matches our learned token `new` perfectly.

The Final Output: When we pass the string through our trained tokenizer, it successfully outputs the exact sequence seen at the top of our diagram: `[low, low, e, s, t, low, e, s, t, new, new]`

This perfectly illustrates the power of BPE: highly frequent words (`low`, `new`) are represented efficiently as single tokens, while rarer word forms (`lowest`) are safely broken down into recognizable sub-words and characters without causing an "Out of Vocabulary" error.

5. Advancing the Frontier: Byte-Level BPE (BBPE)

While standard BPE is powerful, it still has a vulnerability: multi-lingual text and rare characters. If a standard BPE model trained purely on English encounters Japanese Kanji or a specialized emoji, it will fail because those base characters were never in its initial vocabulary.

To solve this, modern systems (like the architecture behind GPT and LLaMA) use Byte-Level Byte-Pair Encoding (BBPE) (Radford *et al.*, 2019; Wang, Cho and Gu, 2020).

Instead of initializing the base vocabulary with Unicode characters (which contains over 140,000 possibilities), BBPE initializes the base vocabulary with raw data bytes. Because all text in modern computing is encoded in UTF-8, every possible character can be represented by a sequence of bytes.

- A standard ASCII character like 'H' takes 1 byte (Hex: `\x48`, Binary: `01001000`).
- A complex emoji or foreign character might take 3 or 4 bytes.

By using the 256 possible byte values as the base vocabulary, the BBPE algorithm guarantees that it can process **any** string of text, in any language, without ever needing an

<UNK> token. The algorithmic merge process remains exactly the same as BPE, but it operates on hexadecimal bytes rather than readable letters.

Example: Standard BPE vs Byte-Level BPE in Action

Imagine a standard BPE model trained *exclusively* on a dataset of classic 19th-century English literature. Its base vocabulary only contains standard ASCII letters, numbers, and basic punctuation.

Now, imagine we feed this trained model a modern text message: "Hi 👋"

Scenario A: Standard Character-Level BPE (The Failure)

The standard BPE tokenizer reads the string character by character:

1. H -> Exists in vocabulary.
2. i -> Exists in vocabulary.
3. (space) -> Exists in vocabulary.
4. 👋 (Waving Hand) -> FATAL ERROR. This Unicode character was never in the training data. The model cannot process it.

- Final Output: ["Hi", " ", "<UNK>"]
- *Result:* The emoji is replaced by the "Unknown" token. The original information is permanently destroyed and cannot be decoded back to the user.

Scenario B: Byte-Level BPE (The Solution)

BBPE does not look at the shape of the characters; it instantly translates the entire string into raw UTF-8 bytes (represented in Hexadecimal) before doing anything else.

- H -> 48 (1 byte)
- i -> 69 (1 byte)
- (space) -> 20 (1 byte)
- 👋 -> F0 9F 91 8B (A complex emoji takes 4 bytes)

Because BBPE's base vocabulary contains all 256 possible single-byte values (00 through FF), it is a mathematical certainty that F0, 9F, 91, and 8B already exist in its foundational dictionary.

- Final Output: ["Hi", " ", "F0", "9F", "91", "8B"]
- Result*:* Even if the AI has never seen a waving hand emoji in its life and hasn't learned to merge those specific bytes, it doesn't crash. It simply outputs the four individual byte tokens. The data is perfectly preserved, no <UNK> token is used, and the text can be perfectly reconstructed during decoding.

6. Hands-On Implementation: Building a BBPE & LSTM Pipeline

To prove the theoretical concepts discussed above, I engineered a custom Byte-Level BPE tokenizer from scratch in Python, bypassing pre-built black-box libraries. This tokenizer was then integrated with a deep learning Long Short-Term Memory (LSTM) network (Hochreiter and Schmidhuber, 1997) to predict text trained on Lewis Carroll's Alice in Wonderland (Carroll, 1865).

6.1 The Byte-Level Tokenizer Class

```python
class BPETokenizer:
    def __init__(self, num_merges=2000):
        self.num_merges = num_merges
        self.merges = []
        self.vocab = {"<PAD>": 0, "<UNK>": 1}
        self.inverse_vocab = {0: "<PAD>", 1: "<UNK>"}
        self.next_id = 2

    def _get_hex(self, text):
        return [f"{b:02x}" for b in str(text).encode("utf-8")]
```

By initializing the `_get_hex` function, we ensure the tokenizer operates strictly on bytes. The `num_merges` hyperparameter dictates how many iterative fusion steps the algorithm will perform. We also establish a base vocabulary containing reserved IDs for Padding (`<PAD>`) and Unknown (`<UNK>`) tokens, though the `<UNK>` token becomes largely obsolete in a byte-level paradigm.

6.2 The Training Loop (Learning the Merges)

```python
def train(self, corpus):
    print(f"[*] Training Tokenizer ({self.num_merges} merge rules)...")
    print(f"[*] This process takes about 1-2 minutes on Colab. Please wait...")
    bytes_list = self._get_hex(corpus)

    unique_base_bytes = sorted(set(bytes_list))
    for b in unique_base_bytes:
        if b not in self.vocab:
            self.vocab[b] = self.next_id
            self.inverse_vocab[self.next_id] = b
            self.next_id += 1

    for step in range(self.num_merges):
        stats = collections.defaultdict(int)
        for i in range(len(bytes_list) - 1):
            stats[bytes_list[i], bytes_list[i+1]] += 1

        if not stats: break

        best_pair = max(stats, key=stats.get)
        new_token = "".join(best_pair)
        self.merges.append(best_pair)

        self.vocab[new_token] = self.next_id
        self.inverse_vocab[self.next_id] = new_token
        self.next_id += 1

        merged_list = []
        i = 0
        while i < len(bytes_list):
            if i < len(bytes_list) - 1 and bytes_list[i] == best_pair[0] and bytes_list[i+1] == best_pair[1]:
                merged_list.append(new_token)
                i += 2
            else:
                merged_list.append(bytes_list[i])
                i += 1
        bytes_list = merged_list

        if (step + 1) % 200 == 0:
            print(f"-> Learned {step + 1}/{self.num_merges} rules...")

    print(f"[*] Training complete. Final vocabulary size: {len(self.vocab)}\n")
```

This snippet demonstrates the greedy heuristic of BPE. In every iteration, a frequency dictionary (`stats`) tallies adjacent byte pairs. The `max()` function identifies the dominant pair, which is then concatenated into a `new_token` and saved to the `self.merges` list. The `while` loop then performs a sweeping replacement across the entire dataset, physically fusing those bytes together before the next iteration begins

6.3 Result

Due to strict word count limitations for this report, the detailed implementation of the Long Short-Term Memory (LSTM) neural network has been omitted from this document. Please refer to the attached `main.py` file for the complete, heavily-commented deep learning architecture, sliding-window data engineering, and training configurations.

```
================================================
 PREDICTING THE NEXT SENTENCE (INFERENCE)
================================================
Input Seed: 'the white rabbit '
AI generated continuation: 'in a pocket, and was them into thought—everything to alice hastily jumped out of'
Full sentence: 'the white rabbit in a pocket, and was them into thought—everything to alice hastily jumped out of'
```

7. References

Abadi, M. *et al.* (2015) *TensorFlow: Large-scale machine learning on heterogeneous systems*. Available at: https://www.tensorflow.org/ (Accessed: 24 March 2026).

Carroll, L. (1865) *Alice's Adventures in Wonderland*. Project Gutenberg. Available at: https://www.gutenberg.org/files/11/11-0.txt (Accessed: 24 March 2026).

Chollet, F. *et al.* (2015) *Keras*. Available at: https://keras.io (Accessed: 24 March 2026).

Gage, P. (1994) 'A new algorithm for data compression', *C Users Journal*, 12(2), pp. 23-38.

Hochreiter, S. and Schmidhuber, J. (1997) 'Long short-term memory', *Neural Computation*, 9(8), pp. 1735-1780.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. and Sutskever, I. (2019) 'Language models are unsupervised multitask learners', *OpenAI blog*, 1(8), p. 9.

Sennrich, R., Haddow, B. and Birch, A. (2015) 'Neural machine translation of rare words with subword units', *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1715–1725. Available at: https://arxiv.org/abs/1508.07909.

Wang, C., Cho, K. and Gu, J. (2020) 'Neural machine translation with byte-level subwords', *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05), pp. 9154-9160.